Creational Design Patterns Report (Deliverable 1)

Project: Bank Management System | **Course:** Software Engineering — Semester 2

Table of Contents

1. Singleton — DataSourceFactory
2. Singleton — AuthContext
3. Factory Method — TransactionFactory
4. Abstract Factory — DAOFactory
5. Abstract Factory — ScreenFactory

1. Singleton — DataSourceFactory

Identified Problem

The system has 7 DAO classes (`AccountDAO` , `CustomerDAO` , `TransactionDAO` , `TransferDAO` , `LoanDAO` , `InterestPostingDAO` , `OverdraftEventDAO`), each needing a database connection pool to communicate with SQL Server. Without centralised control:

- **Multiple connection pools** would be created if each DAO instantiated its own `HikariDataSource` , exhausting database connections and memory.
- **Configuration duplication** — every DAO would independently read `application.properties` and configure pool parameters.
- **Uncontrolled lifecycle** — without a single owner, there is no clean way to shut down all connections on application exit.

Justification

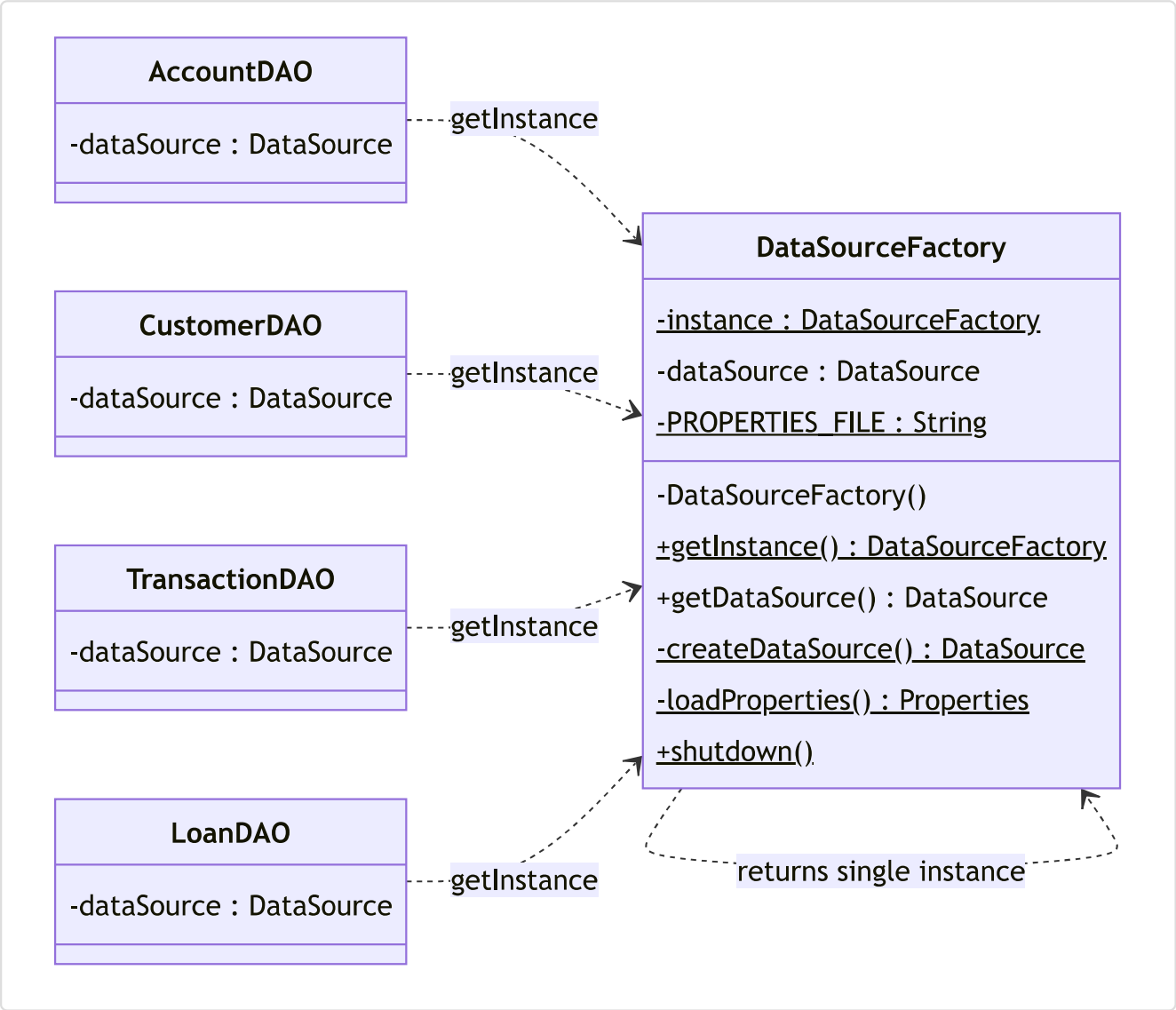
The **Singleton** pattern ensures exactly one `DataSourceFactory` instance (and one connection pool) exists application-wide.

Problem	How Singleton Solves It
Multiple connection pools	One pool is created and shared by all 7 DAOs via `getInstance().getDataSource()`
Configuration duplication	Properties are loaded once in the private constructor
Uncontrolled lifecycle	The static `shutdown()` method closes the single pool on exit

Double-checked locking is used because the application is multi-threaded (JavaFX UI thread + background tasks). It guarantees thread-safe lazy initialization without synchronization overhead on subsequent calls.

Why not other patterns? Factory Method and Abstract Factory create *multiple instances* of different types. The connection pool must be *exactly one shared instance*, making Singleton the correct fit.

UML Class Diagram



2. Singleton — AuthContext

Identified Problem

After login, the customer's identity must be accessible across the entire application — in controllers (to record who performed transactions), in screens (to display the user's name and role), and in navigation logic (to route admins vs. customers). Without centralised session state:

- **Session fragmentation** — if each screen or controller created its own `AuthContext`, they would have different views of who is logged in.
- **Parameter threading** — without a global session holder, the `Customer` object would need to be passed through every method signature.
- **Concurrent access risk** — multiple threads reading/writing session state simultaneously could allow inconsistent states.

Justification

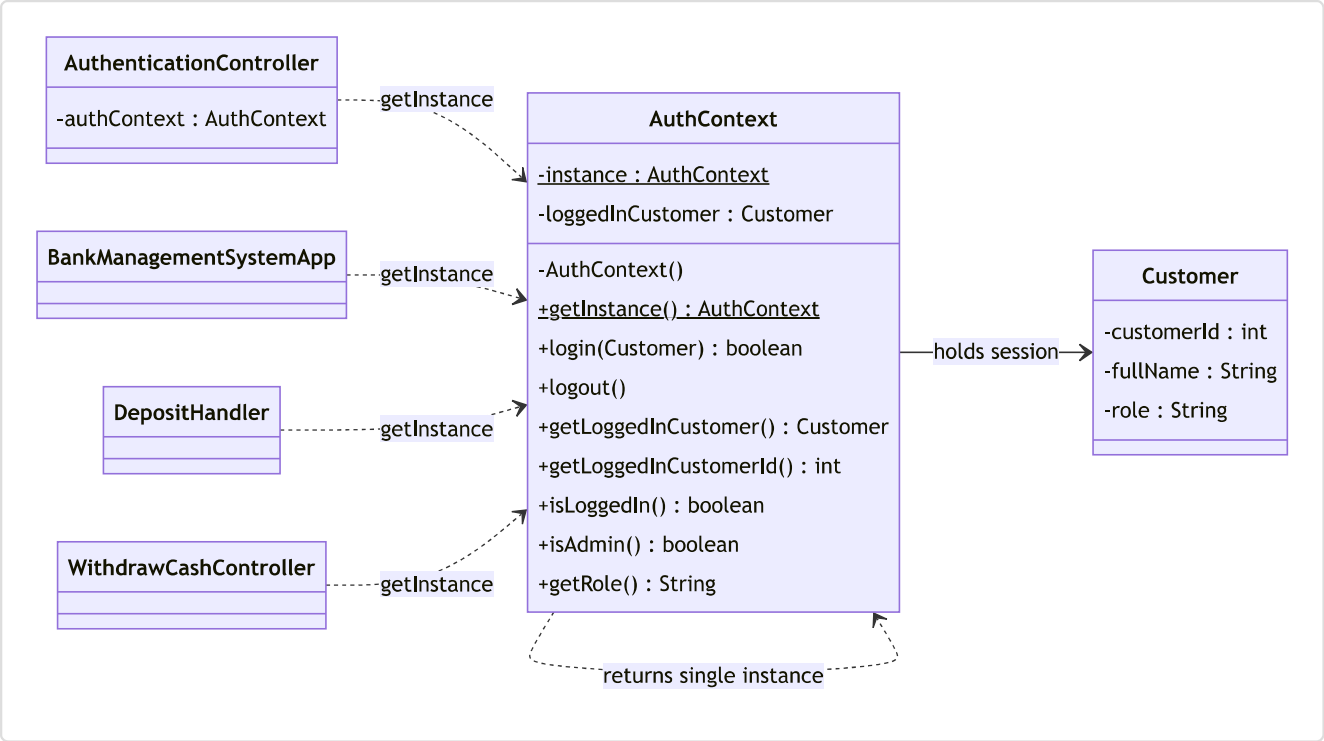
The **Singleton** pattern ensures a single `AuthContext` instance holds the authoritative session state.

Problem	How Singleton Solves It
Session fragmentation	One instance = one source of truth accessible via `getInstance()`
Parameter threading	Any class calls `AuthContext.getInstance().getLoggedInCustomer()` directly
Concurrent access	All mutating methods (`login()` , `logout()`) are `synchronized` on the single instance

Double-checked locking ensures thread-safe initialization. All instance methods are individually `synchronized` because `AuthContext` is mutable (unlike `DataSourceFactory` which is immutable after construction).

Why not other patterns? ThreadLocal would give each thread its own session copy — the opposite of what's needed. Dependency Injection adds framework complexity unnecessary for a JavaFX desktop app with a single session.

UML Class Diagram


AuthenticationController
-authContext : AuthContext
getInstance
BankManagementSystemApp
getInstance
DepositHandler
getInstance
WithdrawCashController
getInstance
AuthContext
-instance : AuthContext
-loggedInCustomer : Customer
-AuthContext()
+getInstance() : AuthContext
+login(Customer) : boolean
+logout()
+getLoggedInCustomer() : Customer
+getLoggedInCustomerId() : int
+isLoggedIn() : boolean
+isAdmin() : boolean
+getRole() : String
returns single instance
holds session
Customer
-customerId : int
-fullName : String
-role : String

3. Factory Method — TransactionFactory

Identified Problem

`Transaction` objects are created in 4 controllers, each for a different use-case:

Controller	Transaction type(s)	Duplicated lines
`DepositHandler`	`"Deposit"`	8 setter calls
`WithdrawCashController`	`"Withdrawal"`	8 setter calls
`TransferHandler`	`"TransferDebit"` , `"TransferCredit"`	2 × 8 setter calls
`MonthlyInterestHandler`	`"InterestPosting"`	8 setter calls

Every creation site repeated the same boilerplate: `new Transaction()` followed by 8 setter calls with hard-coded type strings, type-specific reference-code prefixes (`TXN-` , `INT-`), and default note text. This caused:

- **~40 lines of duplicated code** scattered across 4 classes
- **Hard-coded type strings** risking typos and inconsistencies
- **Type-specific defaults buried in business logic** rather than centralised
- **Difficult extensibility** — adding a new transaction type meant copying the same 8-line block

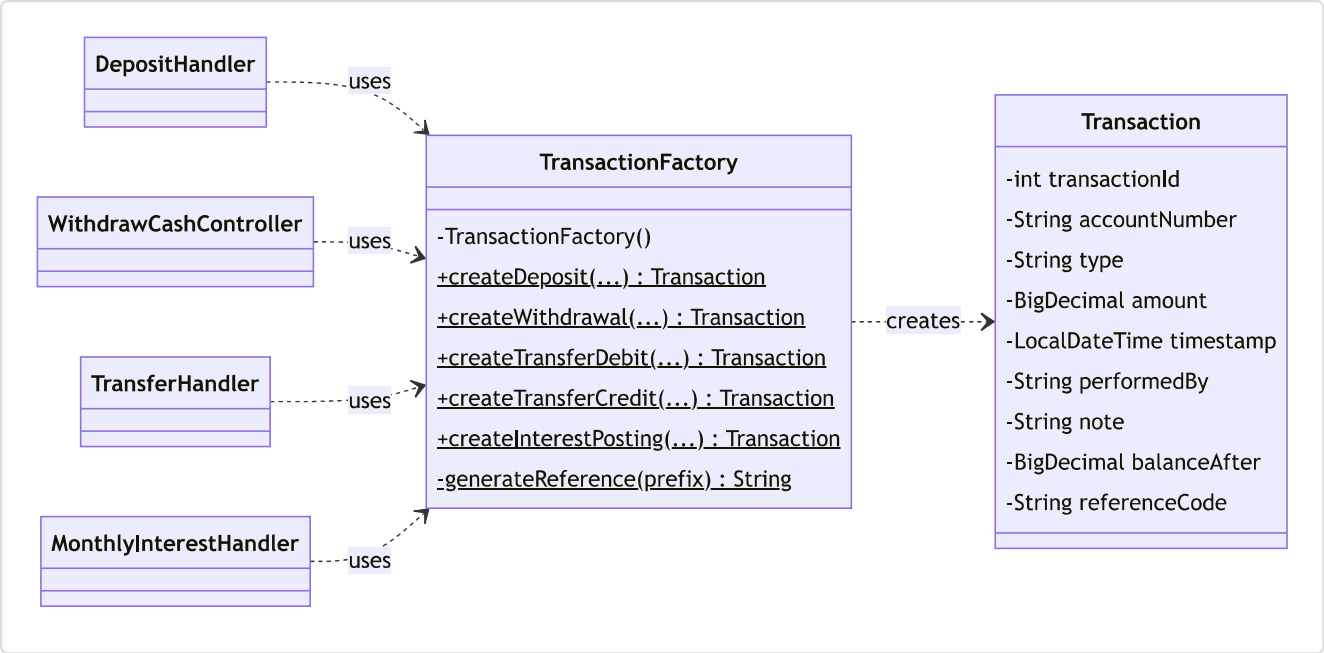
Justification

The **Factory Method** pattern centralises object-creation logic in `TransactionFactory` , with one static factory method per transaction type.

Problem	How Factory Method Solves It
Code duplication	40 lines reduced to 5 one-liner calls
Hard-coded type strings	Defined once inside the factory, not in every controller
Scattered defaults	Reference-code prefixes, notes, and performer values centralised
Difficult extensibility	Adding a new type = adding one factory method, no controller changes

Why Factory Method and not Abstract Factory? We are creating variants of a *single product* (`Transaction`), not families of related products. Factory Method is the correct granularity.

UML Class Diagram


DepositHandler
uses
WithdrawCashController
uses
TransferHandler
uses
MonthlyInterestHandler
uses
TransactionFactory
-TransactionFactory()
+createDeposit(...) : Transaction
+createWithdrawal(...) : Transaction
+createTransferDebit(...) : Transaction
+createTransferCredit(...) : Transaction
+createInterestPosting(...) : Transaction
-generateReference(prefix) : String
creates
Transaction
-int transactionId
-String accountNumber
-String type
-BigDecimal amount
-LocalDateTime timestamp
-String performedBy
-String note
-BigDecimal balanceAfter
-String referenceCode

4. Abstract Factory — DAOFactory

Identified Problem

The system has 7 DAO classes instantiated directly via `new` inside 13 controller constructors:

Controller	Hard-coded DAO construction
`DepositHandler`	`new AccountDAO()`, `new TransactionDAO()`
`WithdrawCashController`	`new AccountDAO()`, `new TransactionDAO()`
`TransferHandler`	`new AccountDAO()`, `new TransactionDAO()`, `new TransferDAO()`
`MonthlyInterestHandler`	`new AccountDAO()`, `new TransactionDAO()`, `new InterestPostingDAO()`
`AuthenticationController`	`new CustomerDAO()`, `new AccountDAO()`
`CustomerProfileController`	`new CustomerDAO()`
`AccountBalanceController`	`new AccountDAO()`
`AccountStatusHandler`	`new AccountDAO()`
`LoanApplicationHandler`	`new LoanDAO()`
`LoanDecisionHandler`	`new LoanDAO()`
`LoanStatusController`	`new LoanDAO()`
`TransactionHistoryController`	`new TransactionDAO()`
`OverdraftHandler`	`new OverdraftEventDAO()`

This caused:

- **Tight coupling** — every controller is directly dependent on concrete SQL Server DAO classes
- **Untestable** — controllers cannot be unit-tested without a real database
- **Inconsistency risk** — no mechanism to guarantee all DAOs come from the same persistence backend

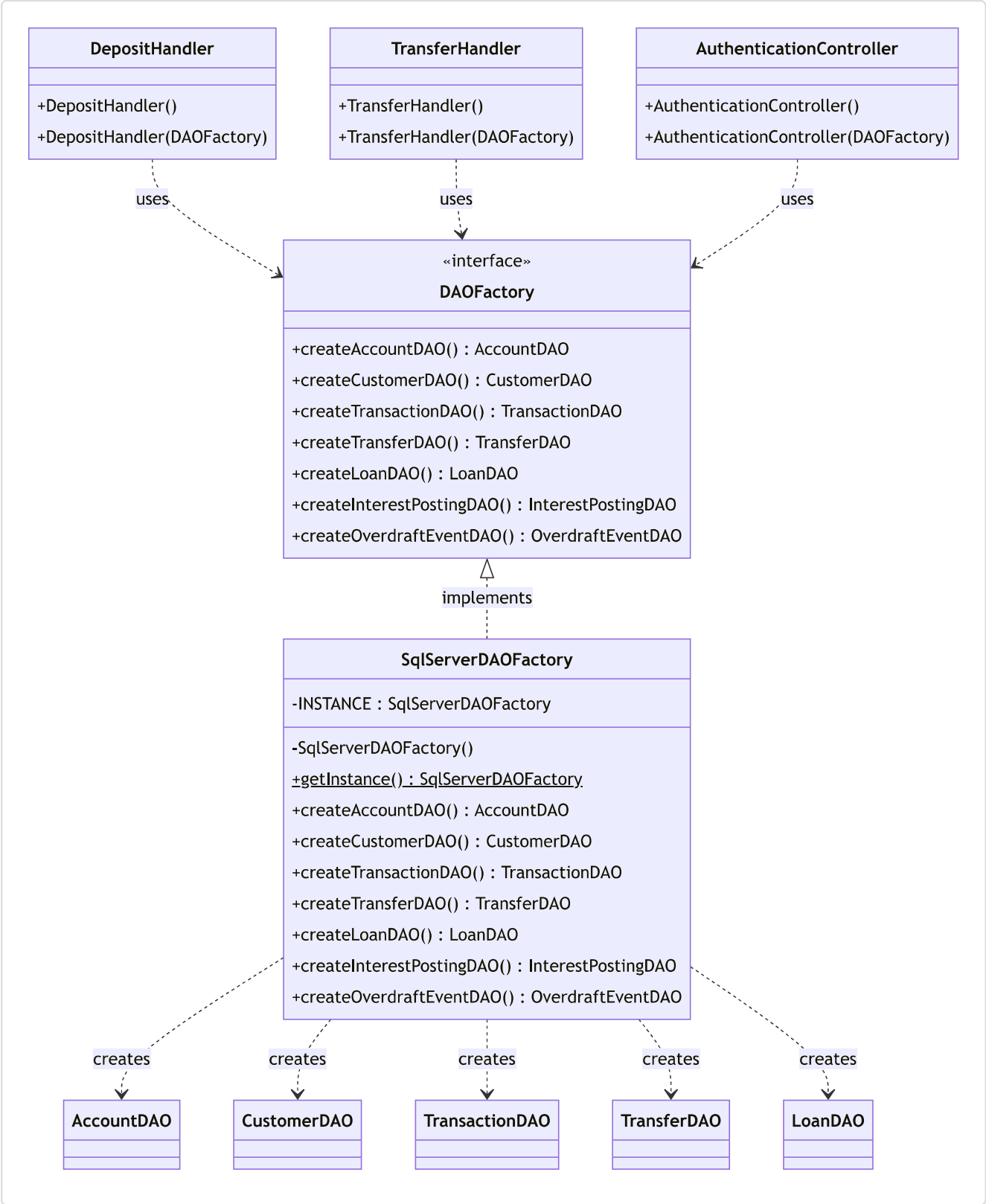
Justification

The **Abstract Factory** pattern provides an interface (`DAOFactory`) with creation methods for every DAO. A concrete factory (`SqlServerDAOFactory`) returns matching implementations.

Problem	How Abstract Factory Solves It
Tight coupling	Controllers depend on the `DAOFactory` interface, not concrete classes
Untestable	An `InMemoryDAOFactory` can be injected for testing with stub DAOs
Inconsistency risk	The factory guarantees all DAOs come from the same backend

Why Abstract Factory and not Factory Method? Controllers need *families* of 2–3 related DAOs that must all come from the same backend — Abstract Factory is designed for producing families of related objects.

UML Class Diagram


DepositHandler
+DepositHandler()
+DepositHandler(DAOFactory)
TransferHandler
+TransferHandler()
+TransferHandler(DAOFactory)
AuthenticationController
+AuthenticationController()
+AuthenticationController(DAOFactory)
uses
uses
uses
«interface»
DAOFactory
+createAccountDAO() : AccountDAO
+createCustomerDAO() : CustomerDAO
+createTransactionDAO() : TransactionDAO
+createTransferDAO() : TransferDAO
+createLoanDAO() : LoanDAO
+createInterestPostingDAO() : InterestPostingDAO
+createOverdraftEventDAO() : OverdraftEventDAO
implements
SqlServerDAOFactory
-INSTANCE : SqlServerDAOFactory
-SqlServerDAOFactory()
+getInstance() : SqlServerDAOFactory
+createAccountDAO() : AccountDAO
+createCustomerDAO() : CustomerDAO
+createTransactionDAO() : TransactionDAO
+createTransferDAO() : TransferDAO
+createLoanDAO() : LoanDAO
+createInterestPostingDAO() : InterestPostingDAO
+createOverdraftEventDAO() : OverdraftEventDAO
creates
creates
creates
creates
creates
AccountDAO
CustomerDAO
TransactionDAO
TransferDAO
LoanDAO

5. Abstract Factory — ScreenFactory

Identified Problem

In `BankManagementSystemApp`, 15 presentation screens are constructed directly with `new` across multiple methods:

Method	Screens created with `new`
`start()`	`LoginScreen`, `AccountSelectionScreen`, `AccountBalanceScreen`, `TransactionHistoryScreen`, `AdminDashboard`
`setupNavigationCallbacks()`	`CreateCustomerProfileForm`, `DepositCashForm`, `WithdrawCashForm`, `UpdateAccountStatusForm`, `LoanReviewForm`, `MonthlyInterestJob`, `OverdraftAlertView`
`createMenuBar()`	`TransferFundsForm`, `LoanStatusView`, `ApplyForLoanForm`

This caused:

- **Tight coupling** — `BankManagementSystemApp` imports and instantiates all 15 concrete screen classes directly
- **No UI swappability** — the app is locked to JavaFX; switching toolkits requires rewriting the main class
- **Inconsistent families** — admin and customer screens form two role-based families that must be created consistently from the same toolkit

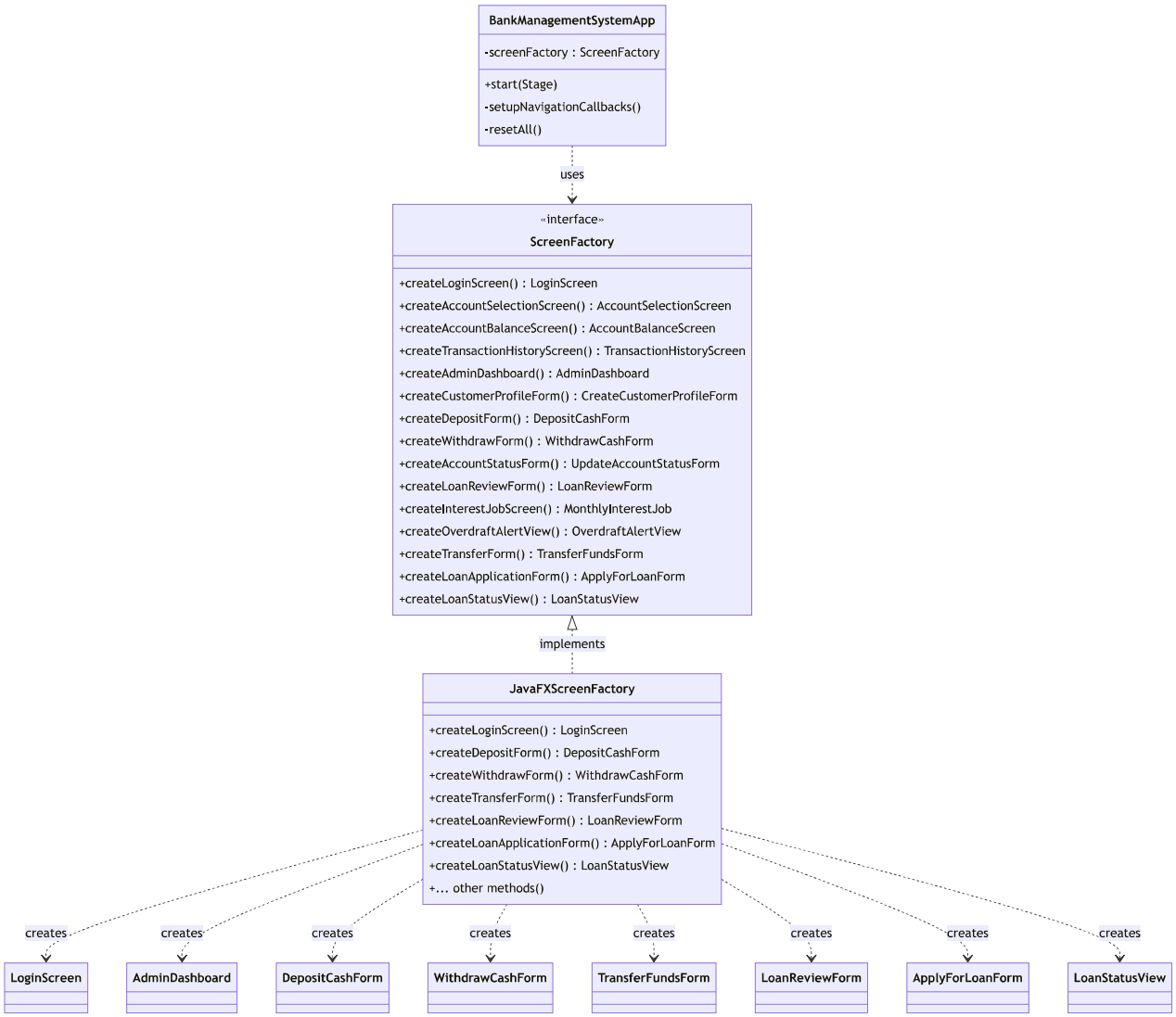
Justification

The **Abstract Factory** pattern provides a `ScreenFactory` interface with creation methods for each screen. `JavaFXScreenFactory` returns current implementations.

Problem	How Abstract Factory Solves It
Tight coupling	`BankManagementSystemApp` depends on `ScreenFactory`, not 15 concrete classes
No UI swappability	A `SwingScreenFactory` could produce alternative implementations without modifying the app
Inconsistent families	The factory guarantees all screens come from the same UI toolkit

Why Abstract Factory and not Factory Method? The application needs *families* of related screens (admin + customer), all from the same toolkit — exactly what Abstract Factory provides.

UML Class Diagram


BankManagementSystemApp
-screenFactory : ScreenFactory
+start(Stage)
-setupNavigationCallbacks()
-resetAll()
uses
«interface»
ScreenFactory
+createLoginScreen() : LoginScreen
+createAccountSelectionScreen() : AccountSelectionScreen
+createAccountBalanceScreen() : AccountBalanceScreen
+createTransactionHistoryScreen() : TransactionHistoryScreen
+createAdminDashboard() : AdminDashboard
+createCustomerProfileForm() : CreateCustomerProfileForm
+createDepositForm() : DepositCashForm
+createWithdrawForm() : WithdrawCashForm
+createAccountStatusForm() : UpdateAccountStatusForm
+createLoanReviewForm() : LoanReviewForm
+createInterestJobScreen() : MonthlyInterestJob
+createOverdraftAlertView() : OverdraftAlertView
+createTransferForm() : TransferFundsForm
+createLoanApplicationForm() : ApplyForLoanForm
+createLoanStatusView() : LoanStatusView
implements
JavaFXScreenFactory
+createLoginScreen() : LoginScreen
+createDepositForm() : DepositCashForm
+createWithdrawForm() : WithdrawCashForm
+createTransferForm() : TransferFundsForm
+createLoanReviewForm() : LoanReviewForm
+createLoanApplicationForm() : ApplyForLoanForm
+createLoanStatusView() : LoanStatusView
+... other methods()
creates
creates
creates
creates
creates
creates
creates
creates
LoginScreen
AdminDashboard
DepositCashForm
WithdrawCashForm
TransferFundsForm
LoanReviewForm
ApplyForLoanForm
LoanStatusView